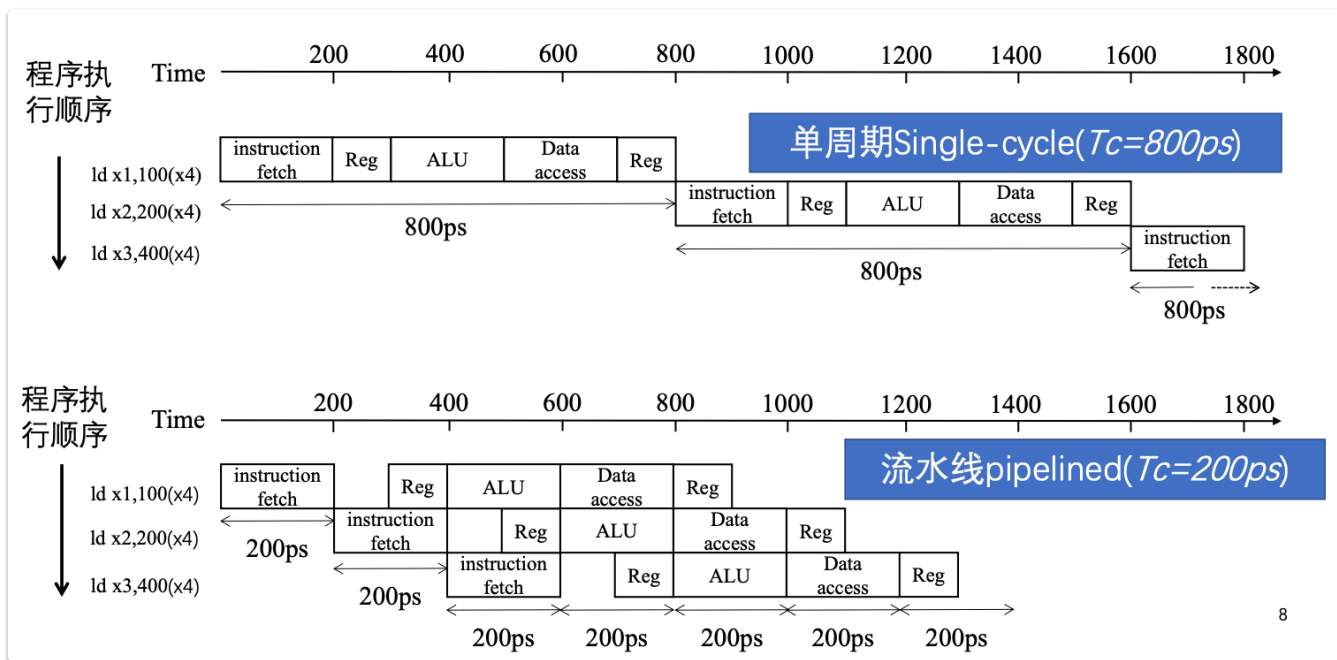


流水线概述

为什么我们需要流水线

对于一次指令的执行，可以分为 5 个阶段：取指 *IF*、译码 *ID*、执行 *EX*、（访存 *MEM*、写入 *WB*）。重叠执行这 5 个阶段可以利用并行性，提高性能。



8

RISC-V: 面向流水线的设计

RISC-V 的特性使其非常适合流水线执行

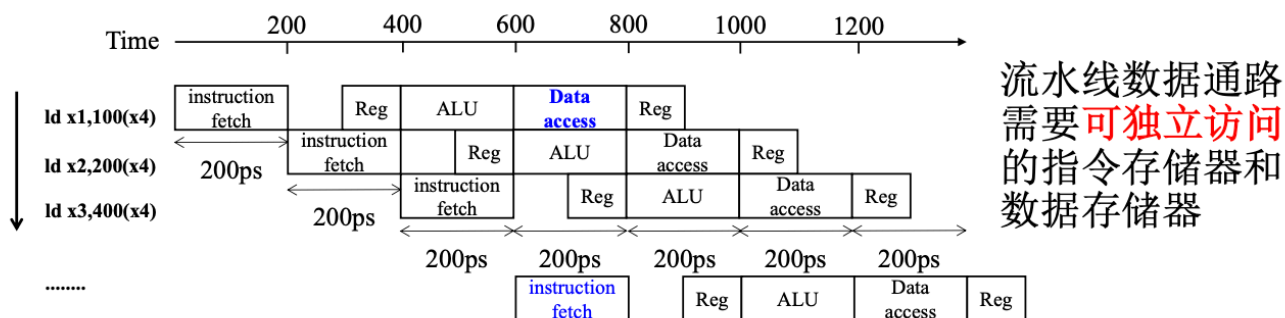
- 指令长度相同：简化取指、译码过程
- 格式整齐：可以在一个阶段完成译码和读寄存器
- 存储器操作只出现在load/store指令中：利用执行阶段来计算存储器地址，然后在下一阶段访问存储器

结构冒险

指因缺乏硬件支持而导致指令不能在预定时钟周期内正确执行。

- 假设下图所示流水线结构 **只有一个存储器(指令和数据共用)**

- load/store需要访问存储器
- 如果有第四条指令，则第一条指令从存储器取数据的同时，第四条指令从同一存储器取指令，流水线发生结构冒险



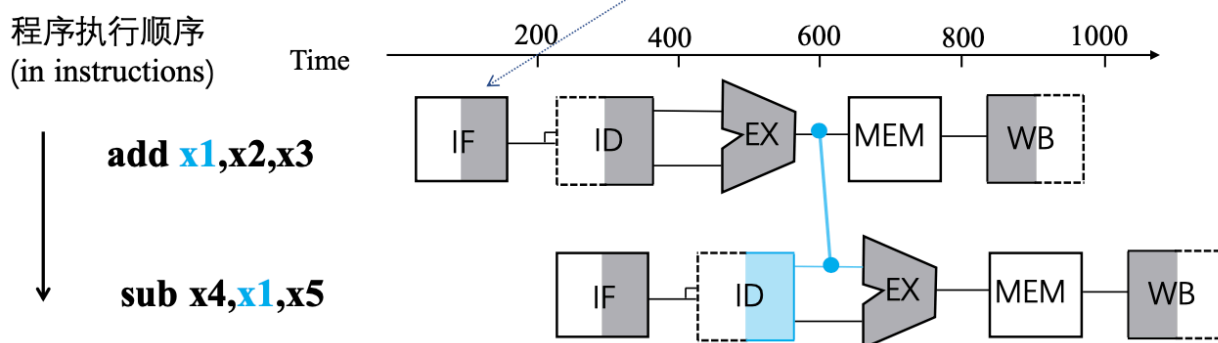
12

数据冒险

一条指令依赖于前面一条尚在流水线中的指令运行结果。解决办法有：

前送 forwarding

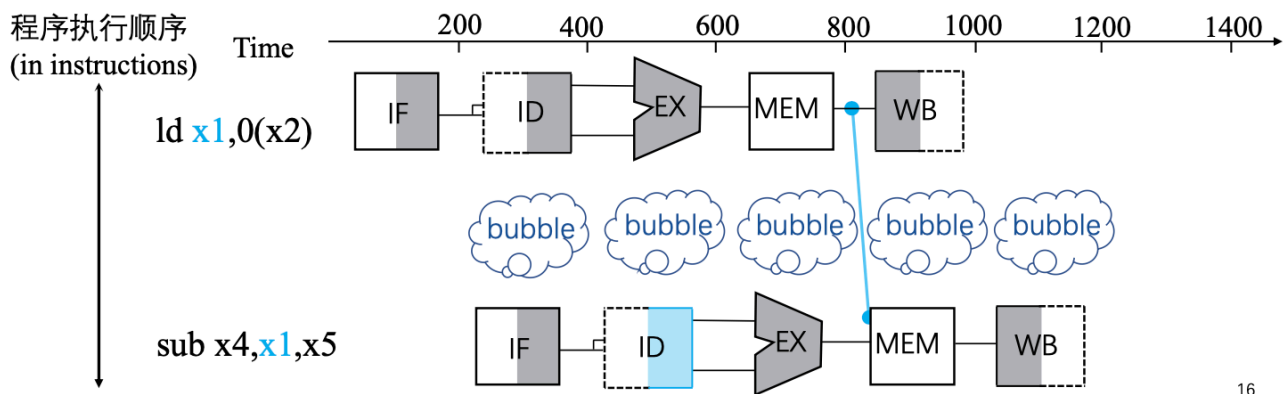
- ALU计算出结果，立刻传数据给后续指令使用，而不是等到数据到达寄存器或存储器
- 需要向内部资源添加额外的硬件
 - 寄存器堆或存储器右半部分为阴影：**读**寄存器堆或存储器
 - 寄存器堆或存储器左半部分为阴影，**写**寄存器堆或存储器



停顿 stall

• 前递不能避免所有的流水线停顿

- 当载入指令要取的数据还没被取回，而后续指令却需要该数据
- 一种解决方案：流水线停顿(stall)，也称为气泡(bubble)，为了解决冒险而实施的一种阻塞



16

控制冒险

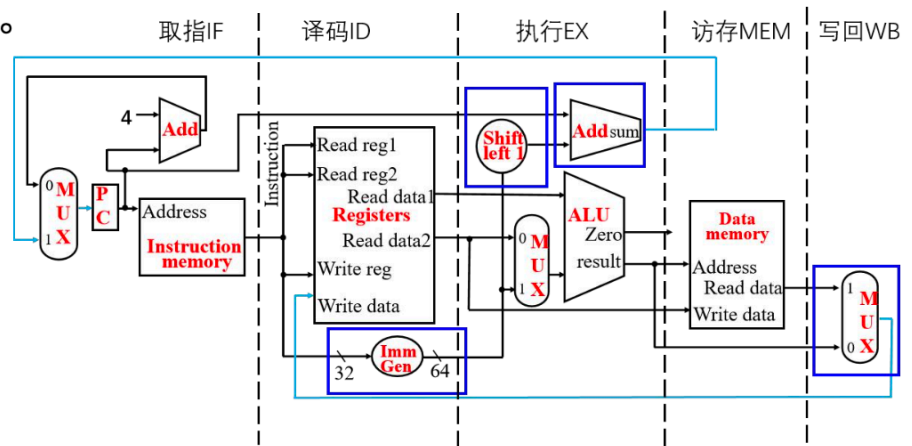
对于分支指令，分支指令后的 IF 阶段需要依赖于分支的结果。如果在取下一个指令前等待分支结果，每个条件分支都会带来停顿，代价过大。所以一般会使用分支预测。在分支指令之后，立即取指，流水线全速前进。分支指令发生跳转时，流水线才会停顿。分支预测方法分为 **静态分支预测** 和 **动态分支预测**

流水线数据通路和控制

数据通路

在构建流水线的数据通路的过程中，我们可以发现，是有可能出现从右到左的数据流向：

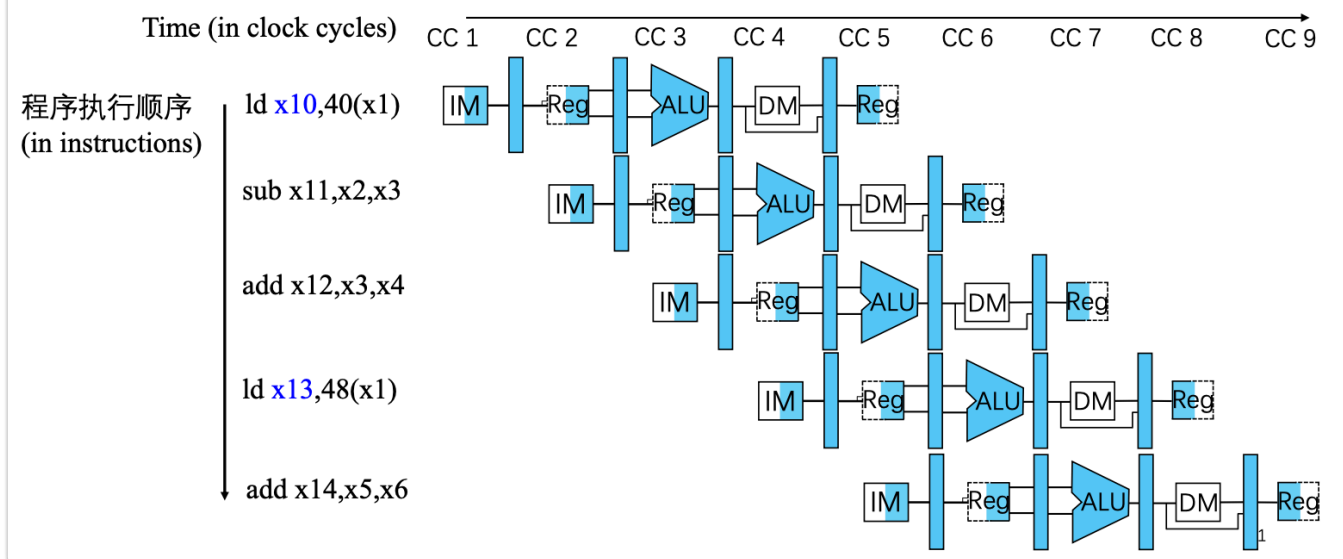
- **写回阶段**：将结果写回位于数据通路中段的寄存器堆
- **下一条指令PC的选择**：在自增PC值与**后面**阶段的分支地址之间进行选择。



从右到左的数据流向可能引发冒险

27

本质是因为流水线之间需要共享数据通路。这个问题可以通过加入 **流水线寄存器** 来解决。通过保存前一个阶段产生的消息，解决指令之间需要同时使用数据通路的情况。

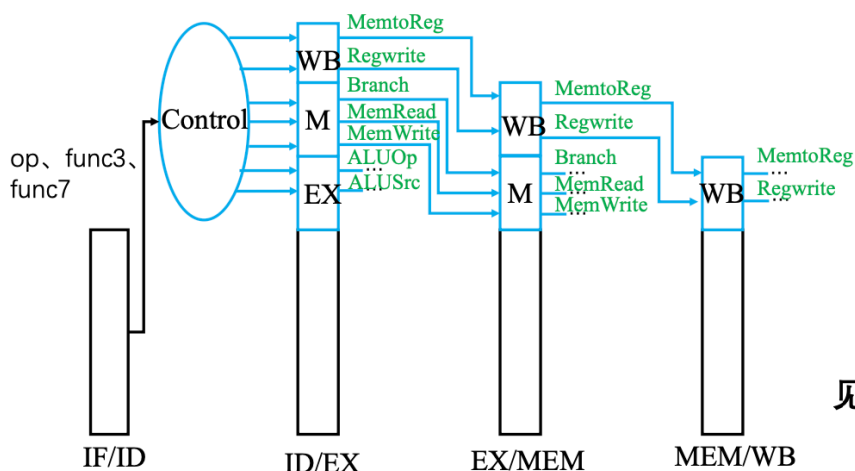


一个简单的例子：比如说在原来，在同一个时钟周期下，指令1 需要往寄存器A写东西，指令2 需要从寄存器A读东西，这会发生冒险。而加入流水线寄存器之后，相当于指令1 的写入会延缓一定时间，让2先读，从而避免了冒险。

- 若要将相关信息从之前的流水线阶段传递到后续的流水线阶段，须将他们放置在流水线寄存器中。
- 在流水线通路设计中的每一个逻辑部件，只能在单个流水线阶段中被使用，否则会发生结构冒险。

流水线控制

- 控制信号从指令中产生，与单周期实现相似
- 根据流水线阶段将控制线分组
 - IF和ID阶段没有需要特别控制的内容



- MemtoReg选择ALU或数据存储器值写入到寄存器堆
- ALUSrc: ALU前的选择器控制信号，用于选择Reg[rs2]或立即数值
- Branch: beq指令的控制信号，相等则分支（备注：本章所讲的数据通路对于分支指令仅实现了beq指令）

见黑书P206-207页的图表

47

流水线控制，后面会用到

解决数据冒险

数据冒险的原因

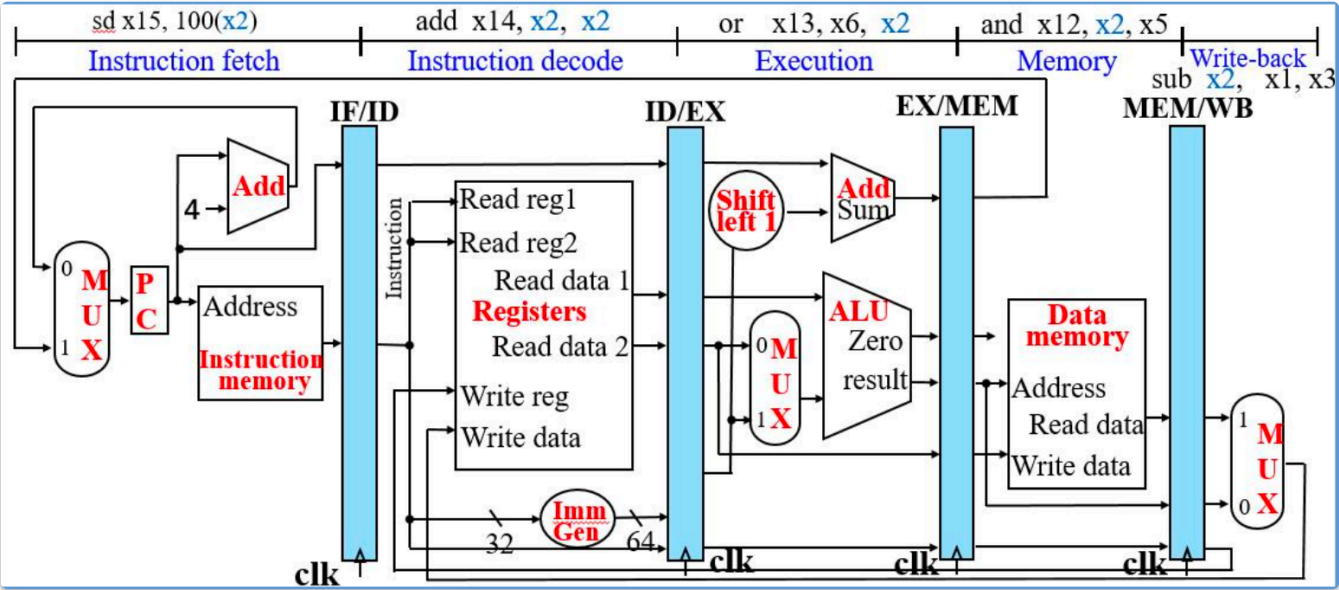
这和寄存器堆的硬件实现有关：一个周期内 **先写后读**，若写操作发生在前半时钟周期，而读操作后半时钟周期，那么读操作会得到本周期内被写入的值。如果这个周期的读操作依赖上个周期的值，此时就会出现数据冒险。

检测方法

如之前说过的，我们可以使用前递或停顿解决这个问题。现在的问题是，我们应该怎么检测是否会发生数据冒险？

命名

先确定一下 **流水线寄存器** 中的具体寄存器命名方式：



比如：

ID/EX.RegisterRs1 表示在 ID/EX 流水线寄存器中，存放寄存器 Rs1 编号的寄存器。

冒险

接着可以来看可能发生的冒险：

- **EX冒险**
 - EX/MEM.RegisterRd == ID/EX.RegisterRs1
 - EX/MEM.RegisterRd == ID/EX.RegisterRs1
 - 即当前指令在 ID 阶段取到了一个源寄存器（Rs1 或 Rs2），但这个值还没有被上一条指令写回（WB）。

比如说像这样

sub x2 , x1,x3 and x12, x2 ,x5	取指令	译码	执行	访存	写回	
		取指令	译码	执行	访存	写回

在同一个时钟周期下，上一条指令还没将结果写到 x2 但下一条指令就开始读 x2，读到的并非其想要的。

MEM冒险

- MEM/WB.RegisterRd == ID/EX.RegisterRs1
- MEM/WB.RegisterRd == ID/EX.RegisterRs2
- 即当前指令依赖的操作数还没有写回寄存器堆，类似 EX 冒险。

sub x2, x1,x3	取指令	译码	执行	访存	写回			
and x12,x2,x5		取指令	译码	执行	访存	写回		
or x13,x6,x2			取指令	译码	执行	访存	写回	

不同的是：是对上上条指令的依赖。

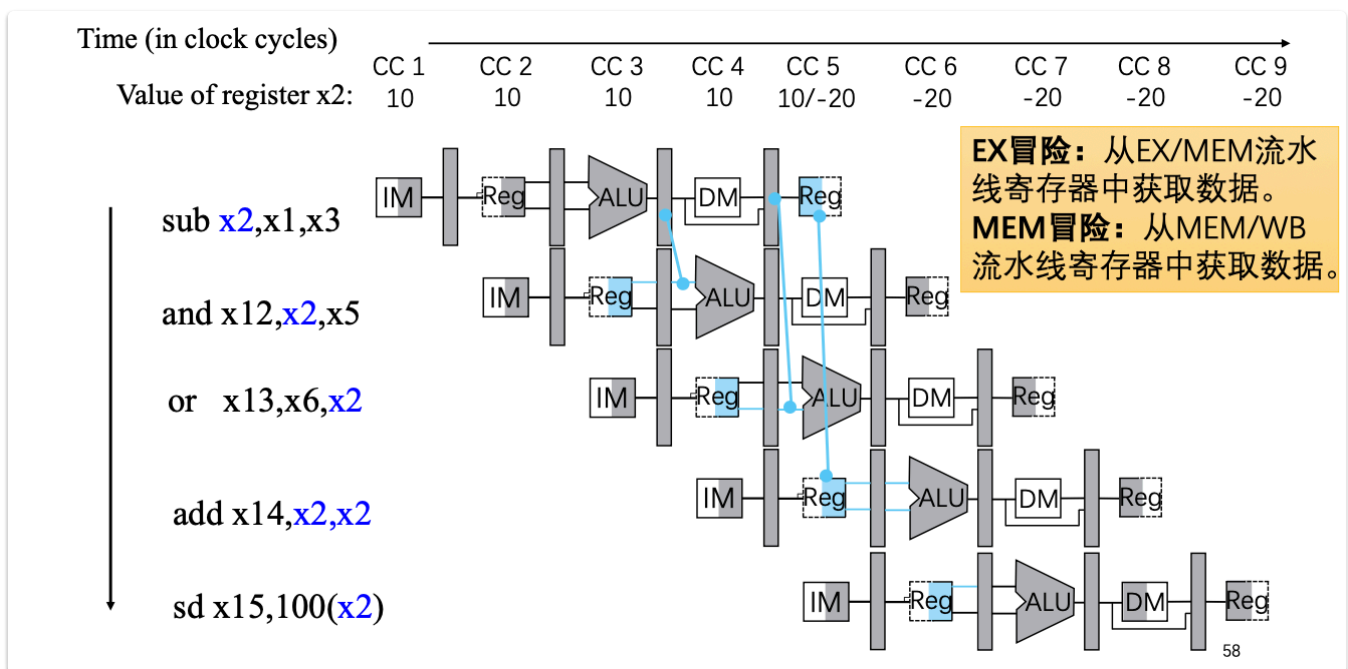
else

同时我们需要考虑其他方面：

- 不是所有指令都需要写回寄存器，通过检查`RegWrite`信号，我们可以判断是否需要写回。
- 如果依赖的是 0号寄存器（永远为0），并不会发生冒险
- 二次冒险

具体设计

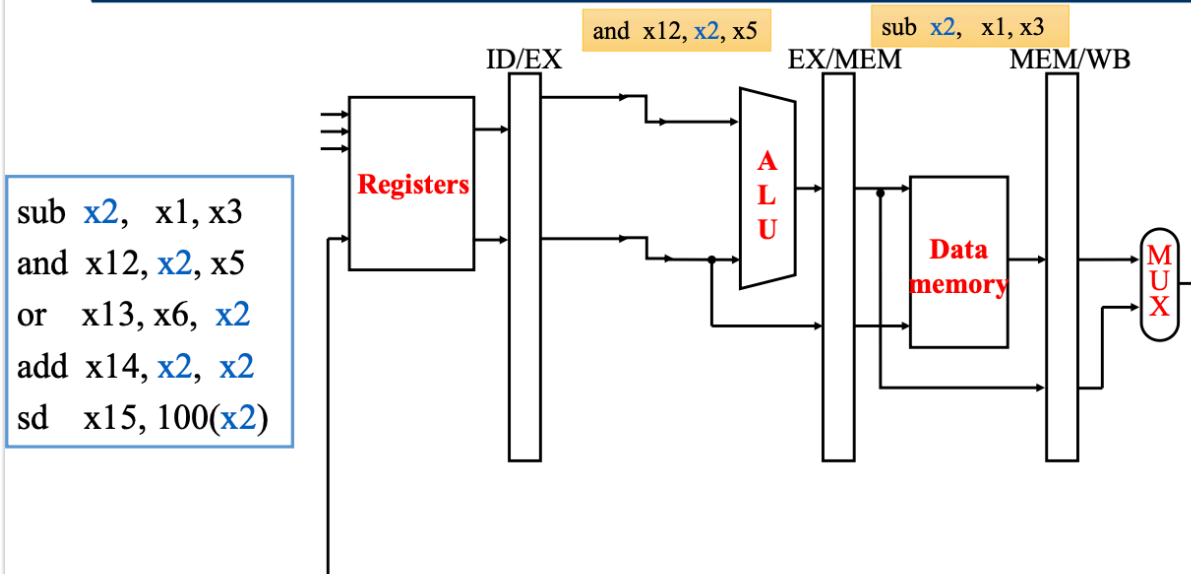
1. 修改依赖



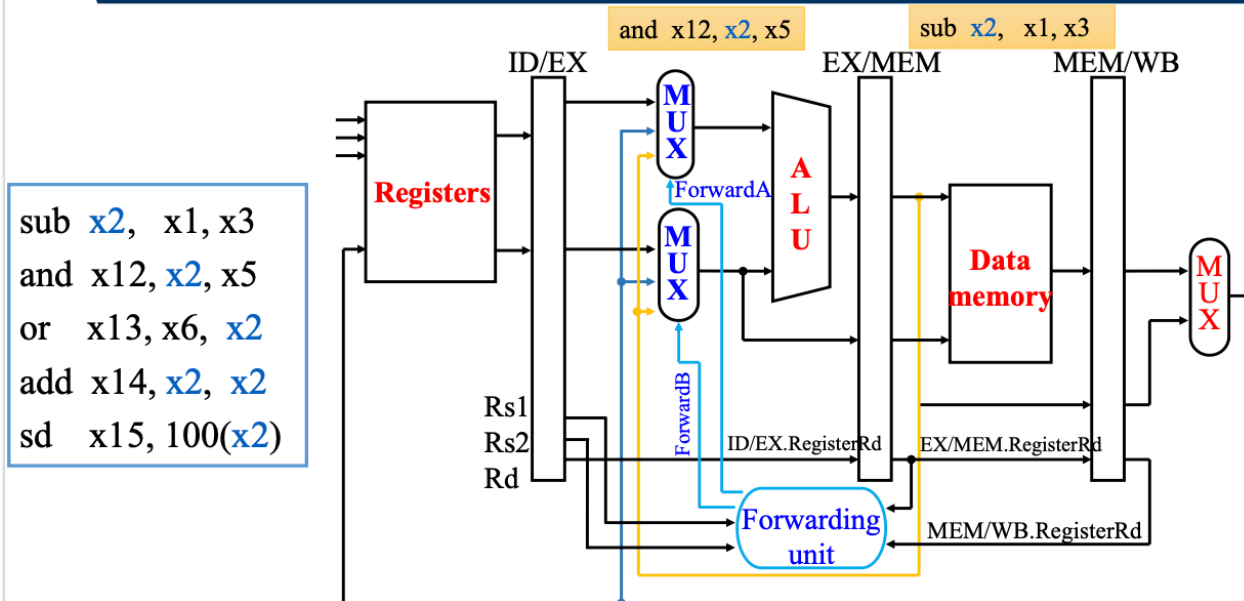
我们需要先修改依赖关系，通过依赖`流水线寄存器`，可以使当前指令可以在前面的指令尚未写回的情况下读到正确的数。

2. 添加硬件

添加前递前



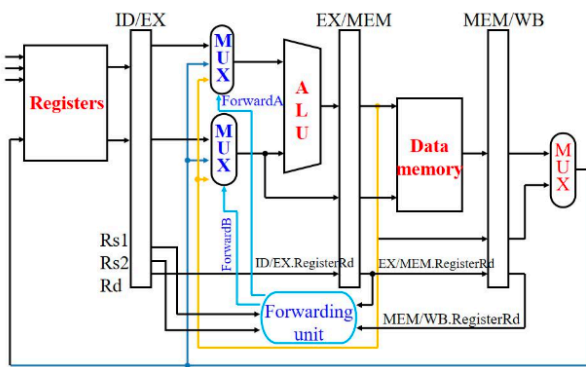
添加前递所需硬件



加入前递之后，Rs1和Rs2字段也要添加到ID/EX流水线寄存器中

Forwarding unit 起到了前递的作用。我们接着来添加前递的判断机制，来控制这个 unit 的工作，决定其是否需要前递，或者从哪前递。

3. 条件判断



ForwardB	数据来源	备注
00	ID/EX 寄存器	ALU的第二个操作数来自寄存器堆
10	EX/MEM 寄存器	ALU第二个操作数来自上一个ALU计算结果的前递
01	MEM/WB 寄存器	ALU的第二个操作数来自数据存储器或者更早的ALU计算结果的前递

ForwardA	数据来源	备注
00	ID/EX 寄存器	ALU的第一个操作数来自寄存器堆
10	EX/MEM 寄存器	ALU的第一个操作数来自上一个ALU计算结果的前递
01	MEM/WB 寄存器	ALU的第一个操作数来自数据存储器或者更早的ALU计算结果的前递

61

EX 冒险的处理:

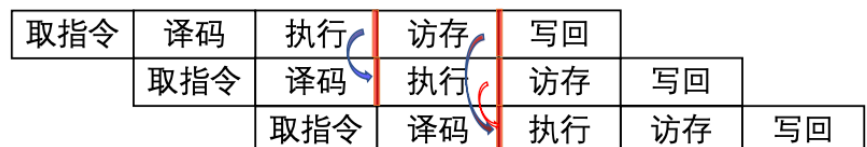
```
# EX 冒险的条件和对应的前递控制
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd == ID/EX.RegisterRs1))
    ForwardA = 10

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd == ID/EX.RegisterRs2))
    ForwardB = 10
```

MEM冒险的处理:

注意 MEM 冒险的条件判断会更复杂, 原因是其需要考虑前两条指令。

```
add x1,x1,x2
add x1,x1,x3
add x1,x1,x4
```



- 存在这种可能: 同时发生了 EX 数据冒险和 MEM 数据冒险。(如上面这种情况👉) 这时, 对于当前指令来说, 应该前递最近的结果 (比如这个例子应该选择 EX 冒险前递)
- 只有 EX 冒险不发生的时候才应该选择 MEM冒险前递

```
# MEM 冒险的条件和相应的前递控制
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and not (EX/MEM.RegWrite
        and (EX/MEM.RegisterRd != 0)
        and (EX/MEM.RegisterRd == ID/EX.RegisterRs1))) # 无EX冒险
```



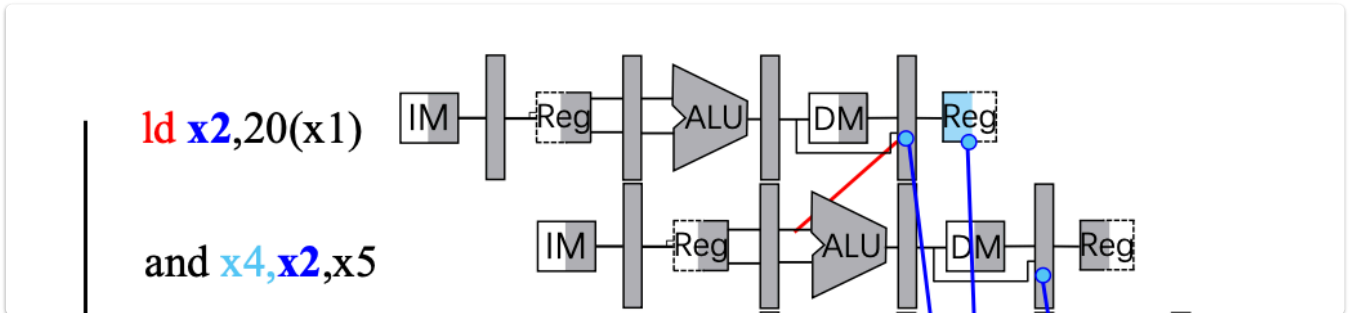
```

and (MEM/WB.RegisterRd == ID/EX.RegisterRs1))
ForwardA = 01

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and not (EX/MEM.RegWrite
        and (EX/MEM.RegisterRd != 0)
        and (EX/MEM.RegisterRd == ID/EX.RegisterRs2)) # 无EX冒险
    and (MEM/WB.RegisterRd == ID/EX.RegisterRs2))
ForwardB = 01

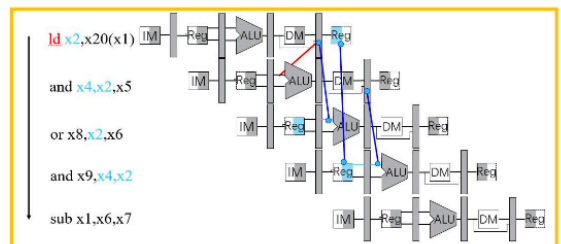
```

载入-使用型数据冒险的处理：



对于这种冒险，无法通过前递完成。我们需要插入气泡，停顿流水线。相当于让当前指令延后一个时钟周期。

- 在ID阶段进行检测
- ALU操作数寄存器字段名称分别是：
 - IF/ID.RegisterRs1, IF/ID.RegisterRs2



- 检测条件
 - ID/EX.MemRead and //是否为load指令？
 $((ID/EX.RegisterRd = IF/ID.RegisterRs1) \text{ or } (ID/EX.RegisterRd = IF/ID.RegisterRs2))$
- 如果检测到这种冒险，停顿流水线（插入气泡）

如何停顿流水线?

被停顿的指令正处于ID阶段:

- 将ID/EX寄存器中控制信号全置为0(RegWrite, MemWrite...)
 - 被停顿的指令在EX, MEM, WB 阶段执行空指令 nop
 - 不会有寄存器或者存储器被写入数据
- 禁止PC寄存器和IF/ID寄存器内容发生改变
 - ID阶段的寄存器会继续使用IF/ID寄存器中相同字段读寄存器
 - 下一条指令会重新取指
 - 1个时钟周期的停顿, 能够让ld指令的MEM阶段完成
 - 就可以把取到的数据前递到EX阶段

- 停顿影响性能, 但保证了正确的结果
- 编译器可以优化代码避免冒险和停顿

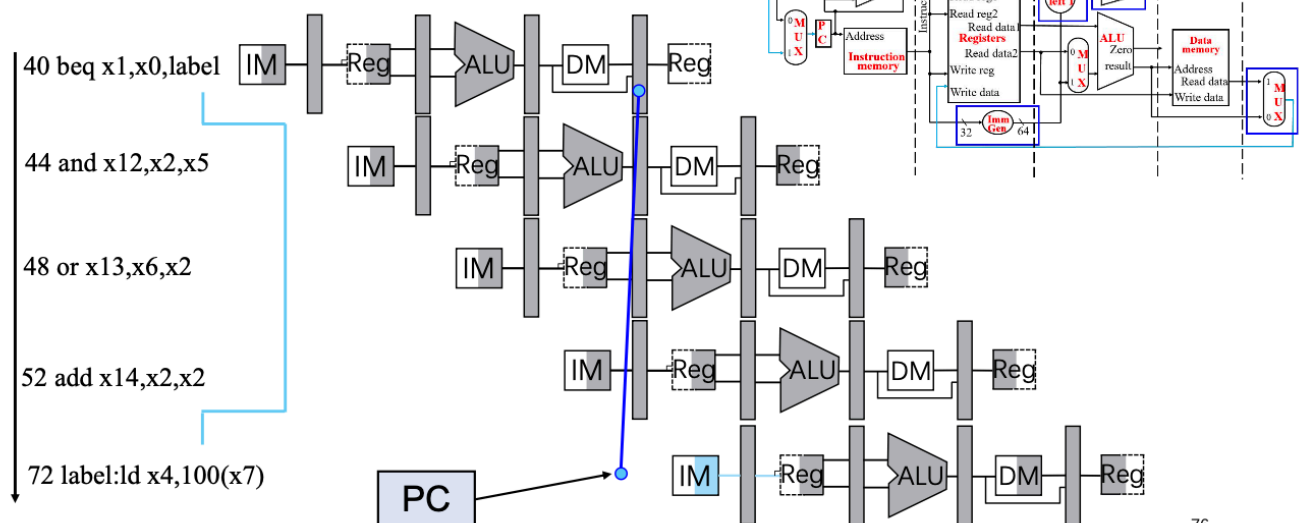
解决控制冒险

分支指令的 **下一条指令**, 其 IF 阶段实际上依赖了分支的结果, 但是其在 IF 阶段时, 分支指令仍在 ID 阶段, 这导致了流水线无法保证取到的是正确的指令。

在 RISC-V 的流水线设计中, 会尽早完成分支判断以及目标地址的计算。通过添加硬件, 可以使这些计算在 ID 阶段完成。

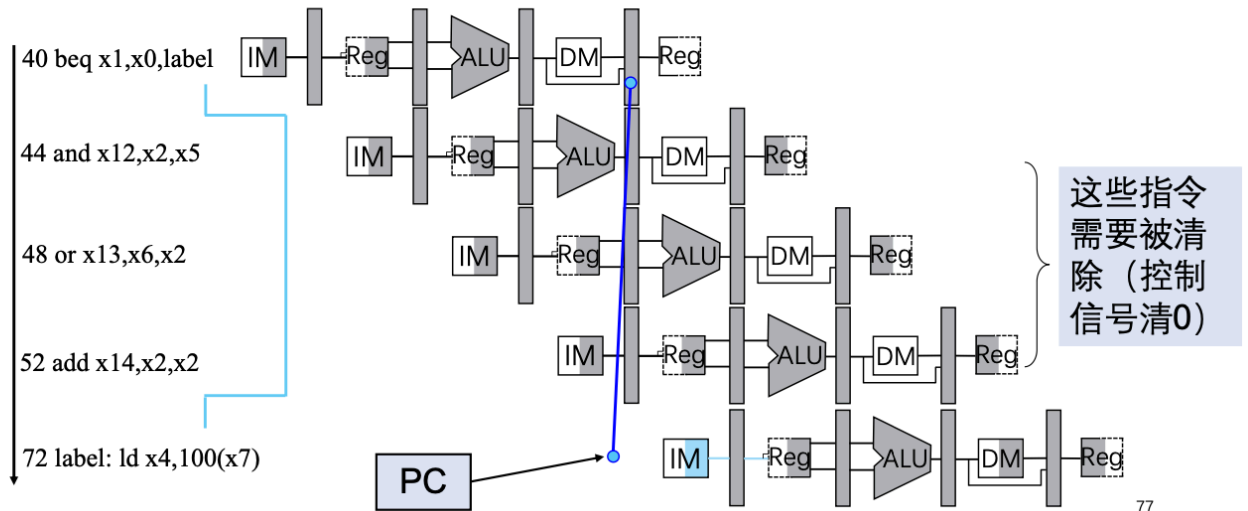
接下来看一下预测失败会发生什么:

- **假设**分支结果在MEM阶段完成确认
 - 分支指令修改PC值发生在MEM阶段



假设默认分支不发生

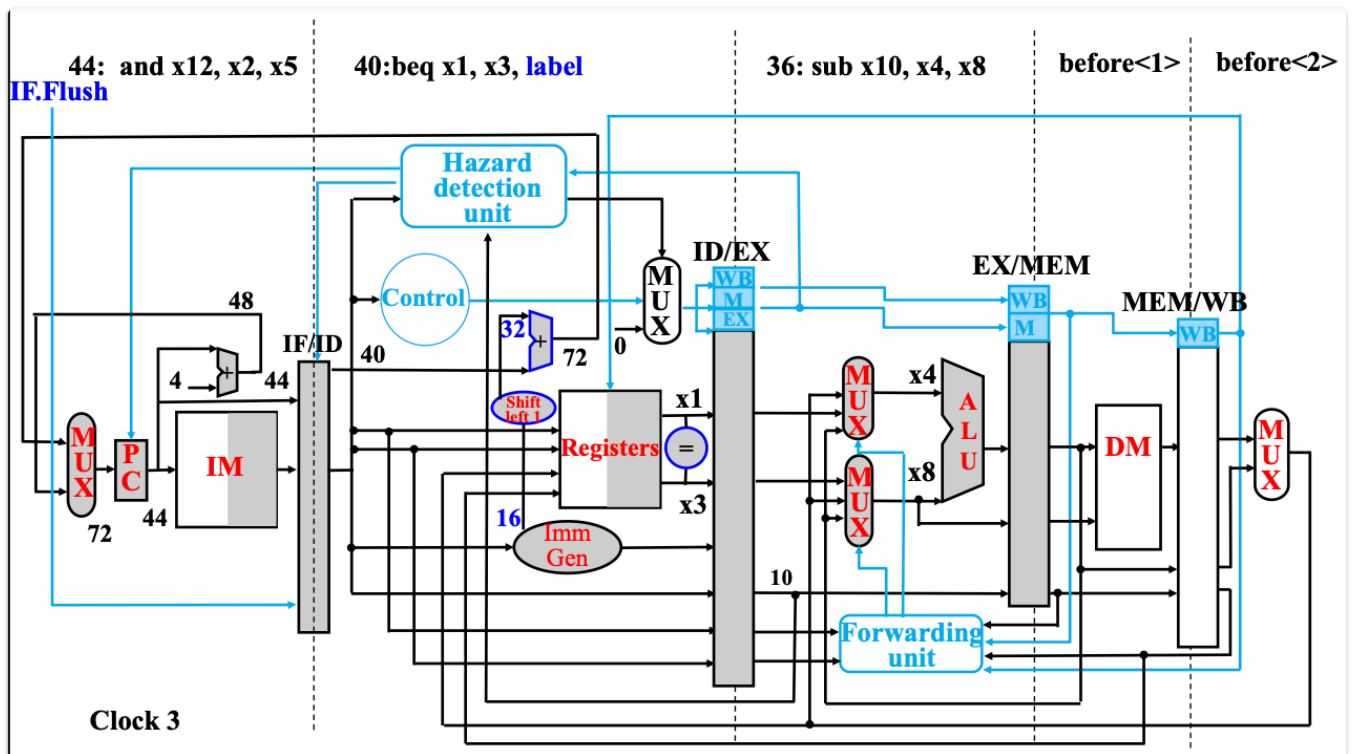
- 如果发生跳转，则需要清除掉分支语句后续指令



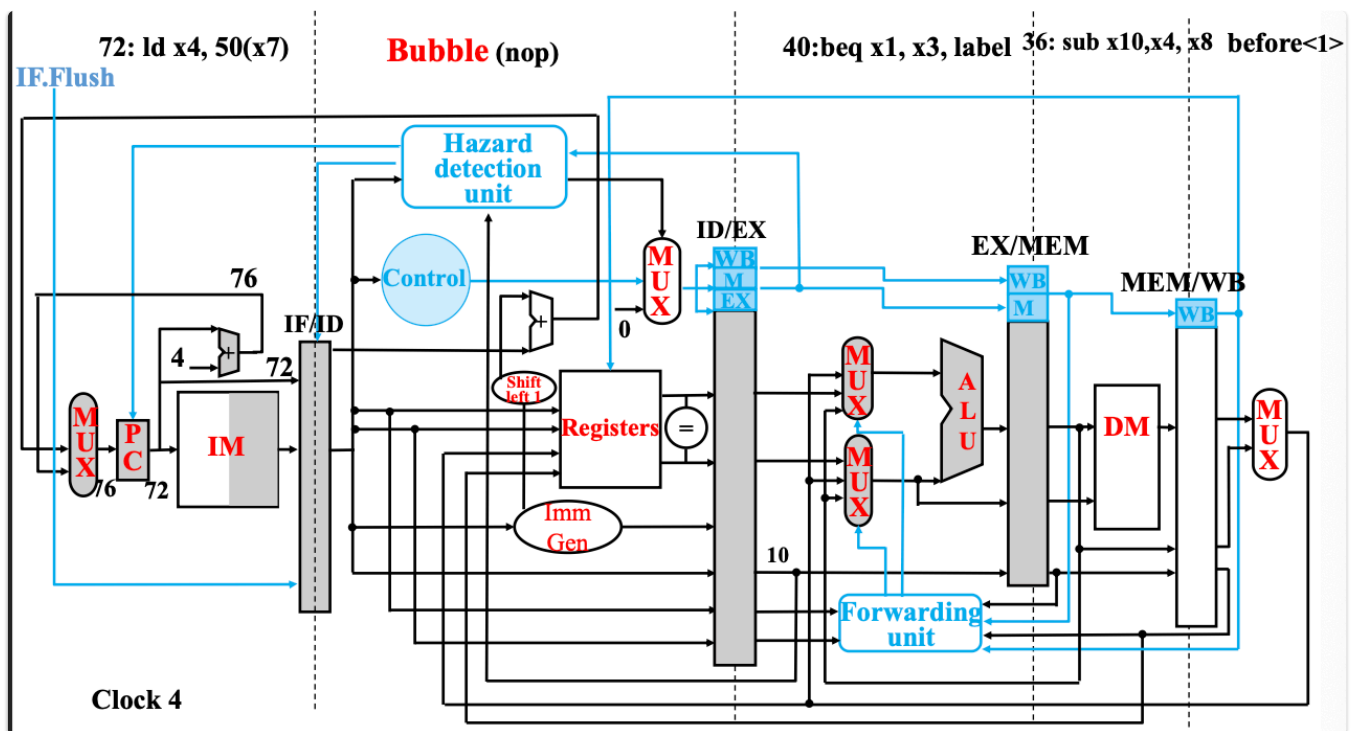
解决办法是：

- 移动硬件，将分支决定提前到 ID 阶段
- 提早判断分支条件
- 提早计算目标分支地址（用 IF/ID 寄存器中的 PC 和 Imm）

我们在 ID 阶段发现需要跳转，会插入气泡，重新取指，如下图：（注意顶端的各阶段对应指令）



在 40 位置的指令在 ID 阶段判断出需要跳转，插入气泡，44 的指令不会进入下一阶段，被清理。



动态分支预测

流水线段数越多，分支预测错误代价越大。我们可以使用动态分支预测：

一种实现方式：采用 **分支预测缓存** 或 **分支历史表 (Branch History Table)**

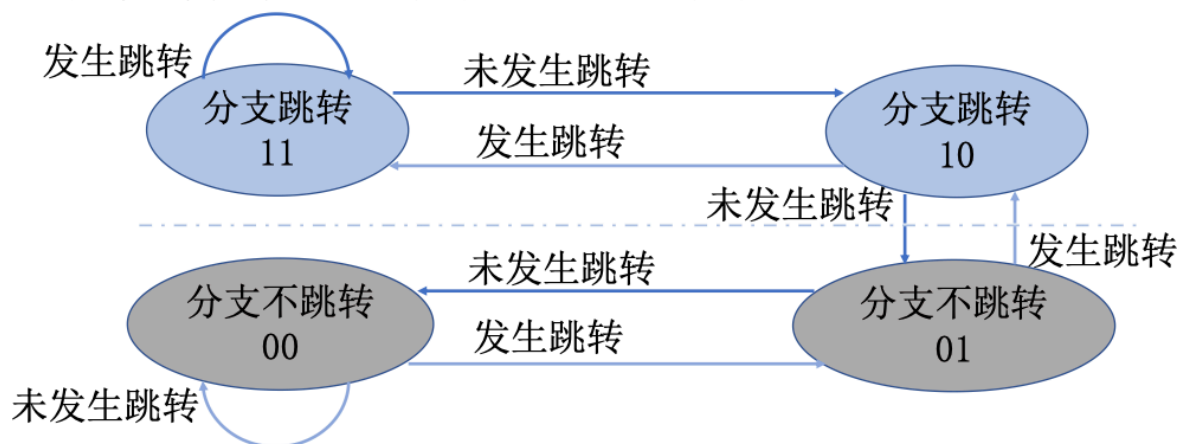
使用分支指令的 **低位地址** 作为索引，每一个表项包含一个或多个位 (bit) 来表明这个分支最近是否发生跳转。

- 1-Bit：预测机制：使用1 位来表示最近是否发生跳转。预测错误时，需要清理错误指令，并修改缓存中的记录（比如说将这一位从 1 变到 0）

缺点：当一个条件分支大部分时间发生跳转，而只有一次不发生跳转，会导致两次预测错误

- 2-Bit：只有发生了连续两次错误时预测才会改变

- 对于左侧两个状态，只有在发生了连续两次错误时预测结果才会被改变（跳转加1，不跳转减1）



在一个分支经常跳转或经常不跳转的情况下（大多数分支都是这样的），只会发生一次预测失败

83

分支目标计算

计算分支目标地址会花费一个时钟周期，这里也可以加入缓存：

使用 **分支指令** 的 PC 值作为索引，缓存 **分支目标** 的 PC 值或者指令

例外

概念

除了分支指令之外，例外和中断也可以改变指令的执行流。不同的指令集对于例外和中断的定义不一样。在 Intel x86 中，两者并无区别。在 RISC-V 中：

- 例外**指**意外的控制流变化**，原因可能是处理器外部or内部
- 中断**仅指**处理器外部事件**引发的控制流变化

事件类型	来源	RISC-V中的表示
系统重启	外部	例外
I/O设备请求	外部	中断
用户程序进行操作系统调用	内部	例外
未定义指令	内部	例外
硬件故障	皆可	皆可

RISC-V下操作系统如何处理例外

- 若执行某指令时出现硬件故障：保存发生例外的指令地址，将控制权转交给操作系统
 - 使用例外程序计数器(Exception Program Counter, EPC)保存发生例外的指令地址
 - 跳转到统一的入口地址，进行例外处理
 - 0000 0000 1C09 0000_{hex}
 - 保存例外发生的原因
 - 例外原因寄存器(Exception Cause Register, CAUSE)
 - CAUSE：大多数位未被使用。

另一种例外处理方式(x86等)

- 采用向量式中断（x86中统称为中断）
 - 例外原因决定后续控制流的起始地址
- 基址寄存器 + 例外原因（作为偏移量）
 - 未定义指令的偏移量： 00 0100 0000₂
 - 硬件故障的偏移量： 01 1000 0000₂
- 转到例外处理程序

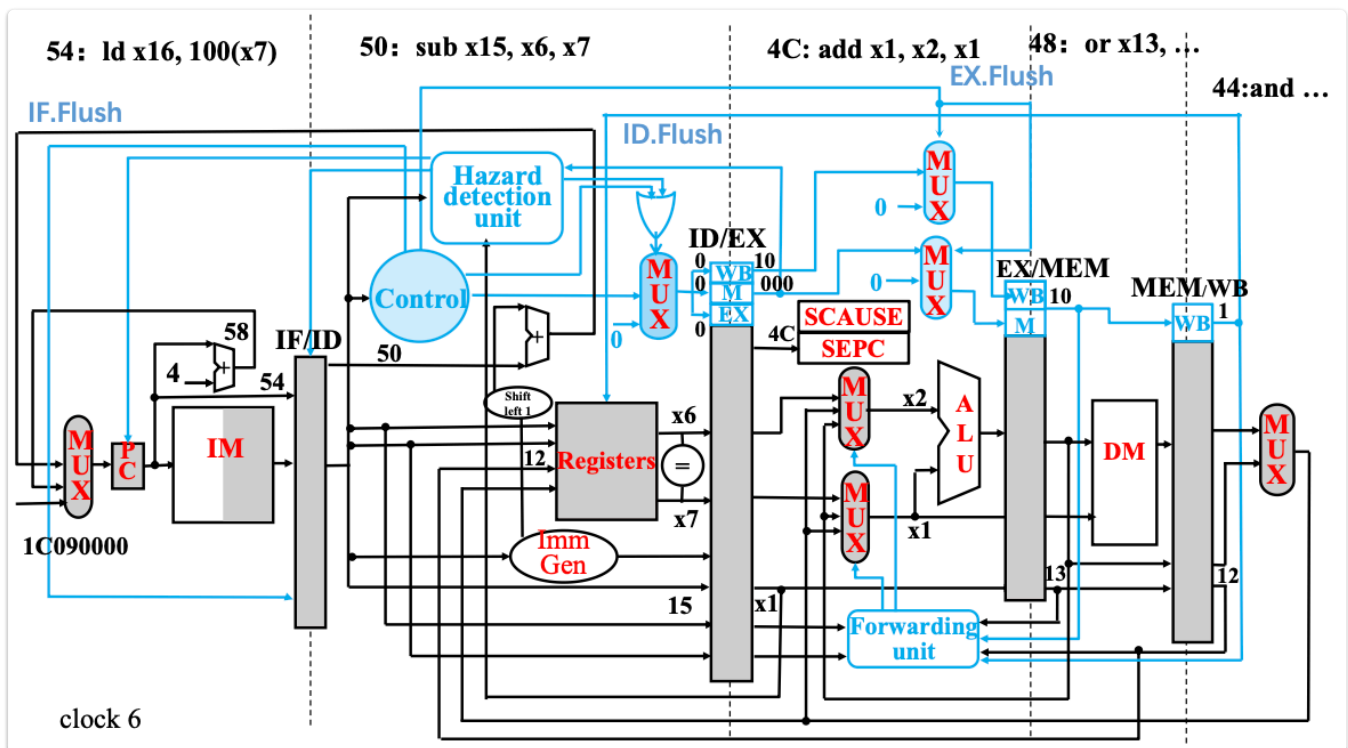
接着交给处理程序处理

例外处理程序的工作

- 如果可以重启程序的执行（I/O设备请求等）
 - 完成例外处理的所有操作
 - 使用SEPC寄存器中的内容重启程序的正常执行
- 否则（未定义指令或硬件故障等）
 - 停止当前程序的执行
 - 使用SEPC, SCAUSE等来报告错误

流水线实现的例外处理

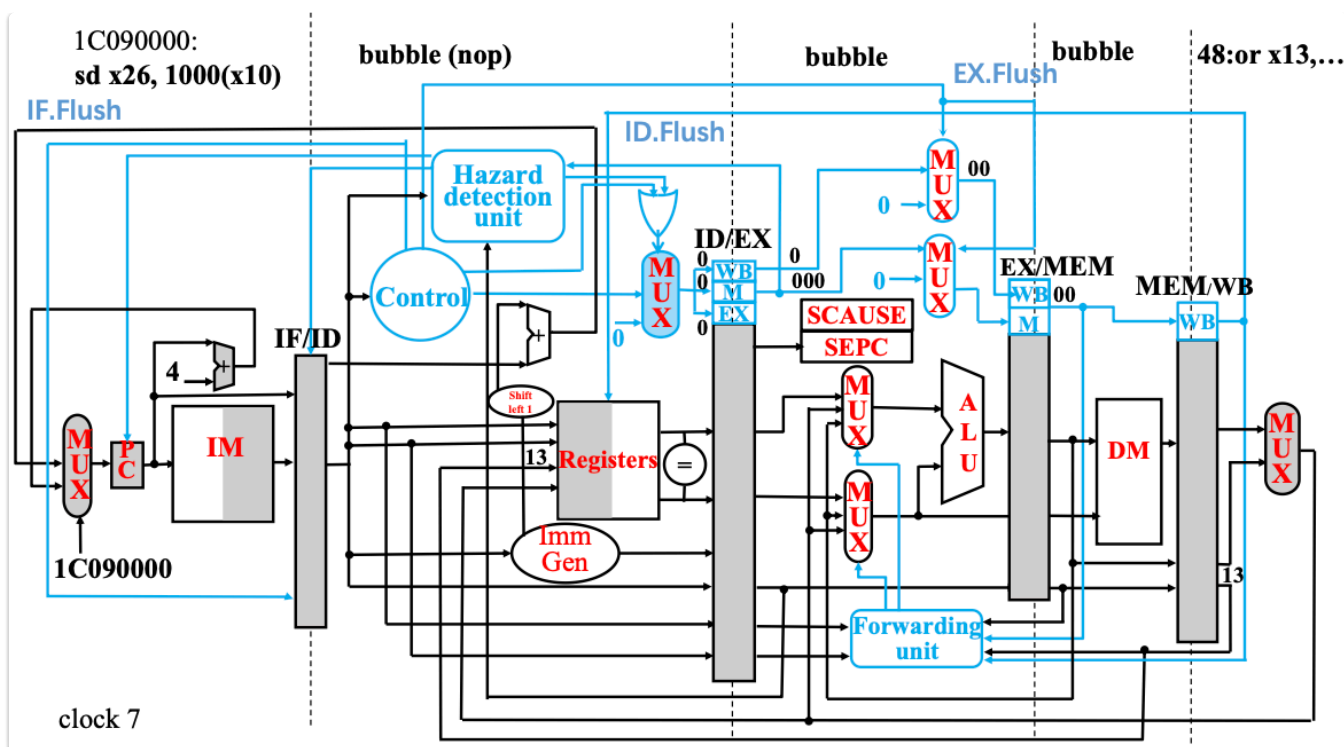
在流水线中，例外处理可以看作另一种控制冒险。举个例子：



我们假设 4C 在 EX 阶段发生了硬件故障，我们需要清理其后面的指令：

- IF阶段的指令：变为nop操作
- ID阶段的指令：增加新的逻辑部件，使译码阶段输出为0

对于 EX阶段的指令：需要保留x1原本的值



- 例外之前的指令，正常完成
- 设置SEPC和SCAUSE的寄存器值
- 转到例外处理程序（1C090000）

计算机体系结构的7个伟大思想

考过，要记下来

1. 使用抽象简化设计
2. 加速经常性事件
3. 通过并行提高性能
4. 通过流水线提高性能
5. 通过预测提高性能
6. 存储层次
7. 通过冗余提高可靠性